

Data Communication

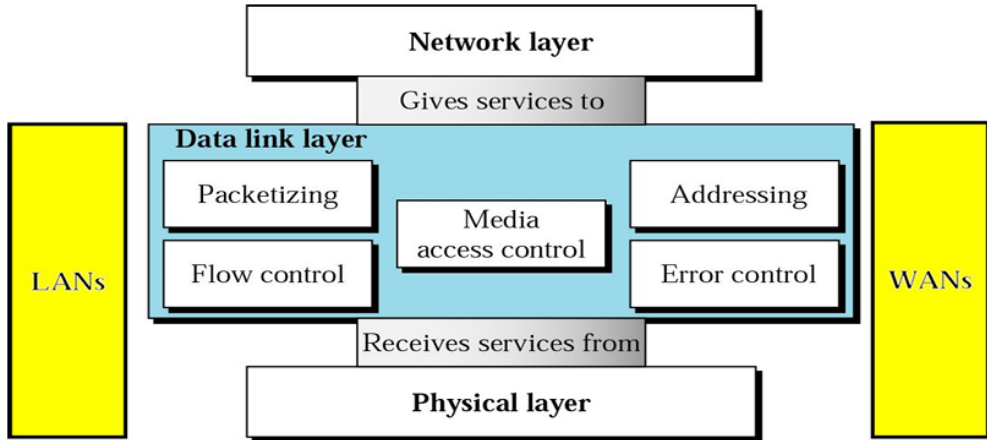
Content 09: Framing, Error Correction & Detection

Abdullah Ibne Masud Mahi (AIMM)

Department of Computer Science and Engineering
United International University



Position of Data Link Layer



Framing

- The data link layer needs to pack bits into **frames**, so that each frame is distinguishable from another.
- **Analogy:** our postal system practices a type of framing – inserting a letter into an envelope separates one piece of information from another; the envelope serves as the **delimiter**.
- Two broad approaches:
 - **Fixed-Size Framing**
 - **Variable-Size Framing**



Framing: Fixed vs. Variable Size

- **Fixed-Size Framing**

- No need to define the boundaries of the frame explicitly – the **size itself** acts as the delimiter.

- **Variable-Size Framing** – needs an explicit way to mark where a frame starts/ends.

Two flavors:

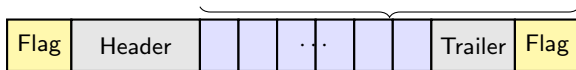
- **Character-Oriented Protocols:** data carried are 8-bit characters from a coding system.
- **Bit-Oriented Protocols:** the data section is a sequence of bits, interpreted by the upper layer as text, graphic, audio, video, etc.



Character-Oriented Protocols

- To separate one frame from the next, an **8-bit (1-byte) flag**, composed of protocol-dependent special characters, is added at the **beginning and end** of a frame.

Data from upper layer (variable # of characters)



Character-Oriented Protocols: Byte Stuffing

- **Problem:** any pattern used for the flag could also legitimately appear as part of the data.
- **Fix:** a **byte-stuffing** strategy.

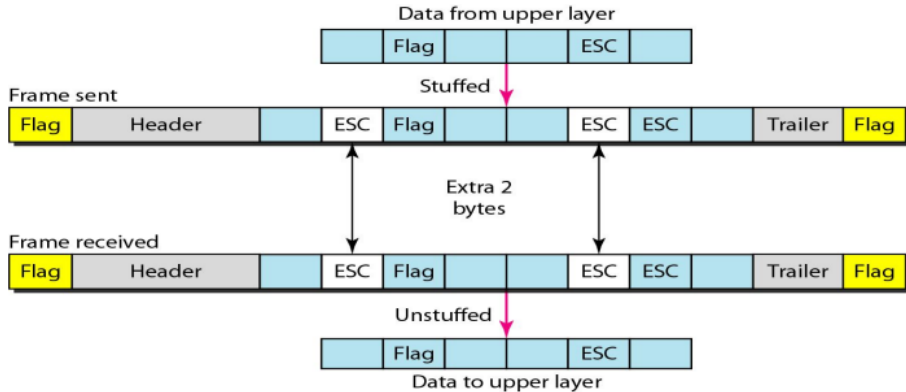
Byte Stuffing

The process of adding **1 extra byte** (an ESC – predefined bit pattern) whenever a flag or escape character occurs **within** the text.

- Receiver: whenever it encounters ESC, it **removes** it and treats the **next** character as plain data, not a delimiting flag.
- If ESC itself appears in the data, it must also be preceded by **another** ESC.

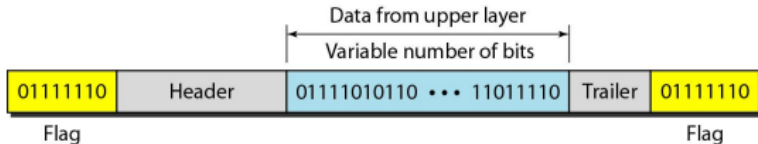


Byte Stuffing and Unstuffing



Bit-Oriented Protocols

- Most protocols use a special **8-bit pattern flag** 01111110 as the delimiter marking the beginning and end of a frame.



Bit-Oriented Protocols: Bit Stuffing

- The 01111110 flag pattern can appear inside the data too – same problem as before.
- Fix: stuff **1 single bit** (instead of a whole byte) – called **Bit Stuffing**.

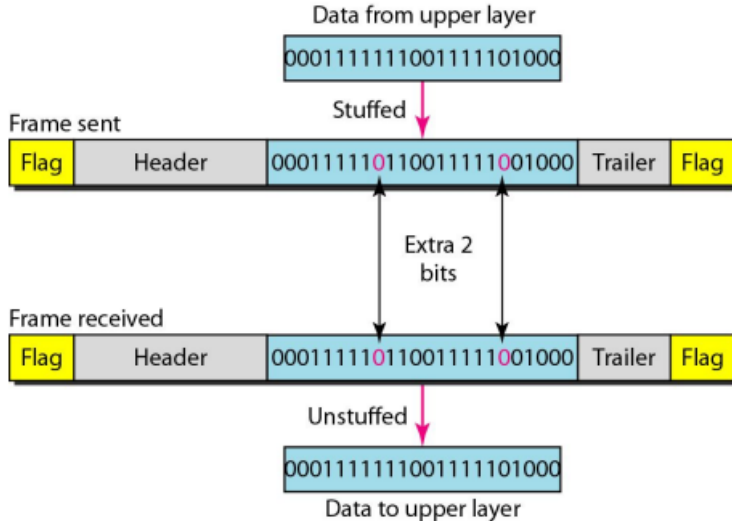
Bit Stuffing

The process of adding one extra 0 whenever **five consecutive 1s** follow a 0 in the data, so the receiver does not mistake the pattern for a flag (0111110).

- The receiver removes this extra stuffed bit before passing data upward.



Bit Stuffing and Unstuffing



Flow and Error Control

- The most important responsibilities of the data link layer are **flow control** and **error control**.
- Collectively, these functions are known as **data link control**.

Flow Control

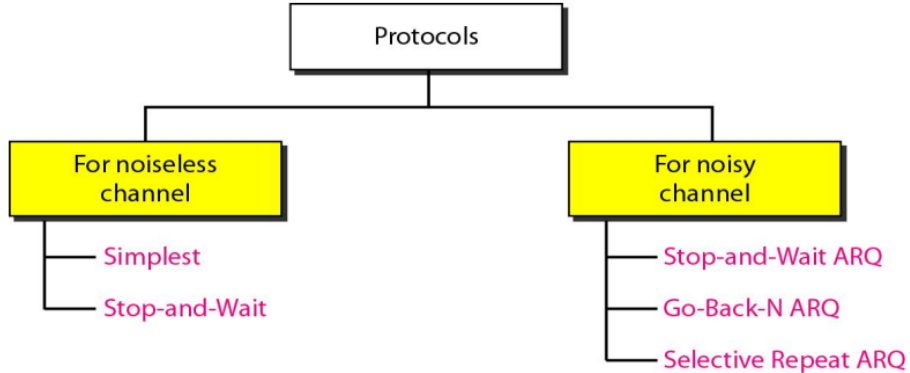
A set of procedures used to restrict the amount of data the sender can send **before** waiting for acknowledgment – regulating the sender's data rate so the receiver's buffer is never overwhelmed.

Error Control

Based on **Automatic Repeat Request (ARQ)** – the retransmission of data. Covers both **error detection** and **error correction**.



Taxonomy of Data Link Protocols



Error Detection & Correction

- **Detection:** looking only to see if any error has occurred.
- **Correction:** need to know the exact number of bits that are corrupted and more importantly, their location in the message.
- The correction of errors is more difficult than the detection.
- Error Detection and Correction are implemented at the data link layer and the transport layer.



Error Detection & Correction

- **Forward Error Correction**

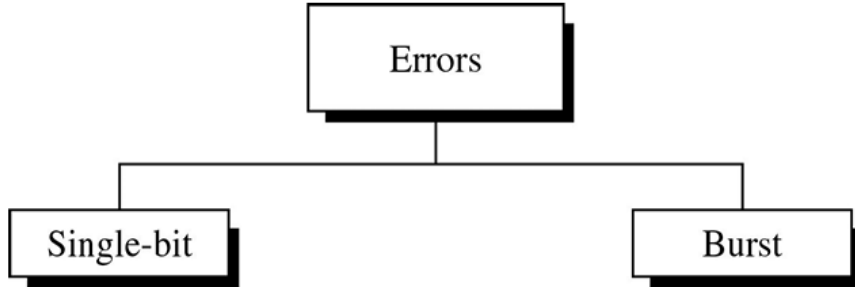
The process in which the receiver tries to guess the message by using redundant bits.

- **Retransmission**

The technique in which the receiver detects the occurrence of an error and asks the sender to resend the message.

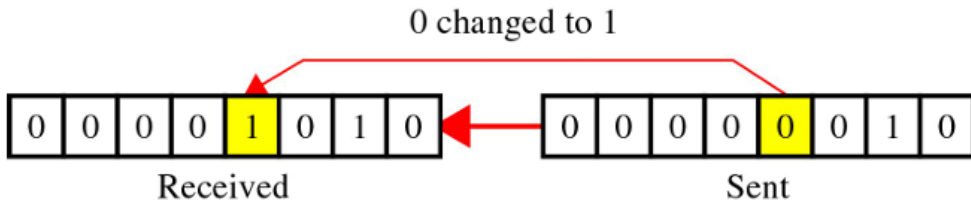


Types of Errors



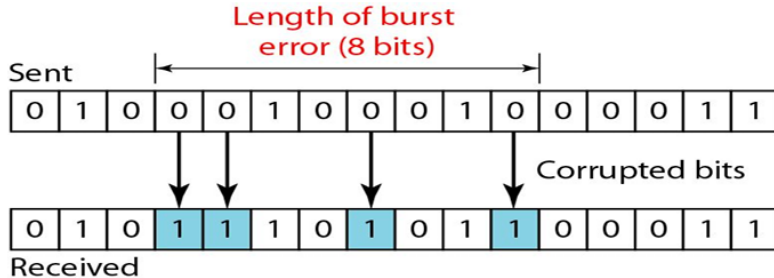
Single-Bit Error

Only 1 bit in the data unit has changed.



Burst-Bit Error

- A burst error means that two or more bits in the data unit have changed.
- The length of the burst is measured from the first corrupted bit to the last corrupted bit. Some bits in between may not have been corrupted.



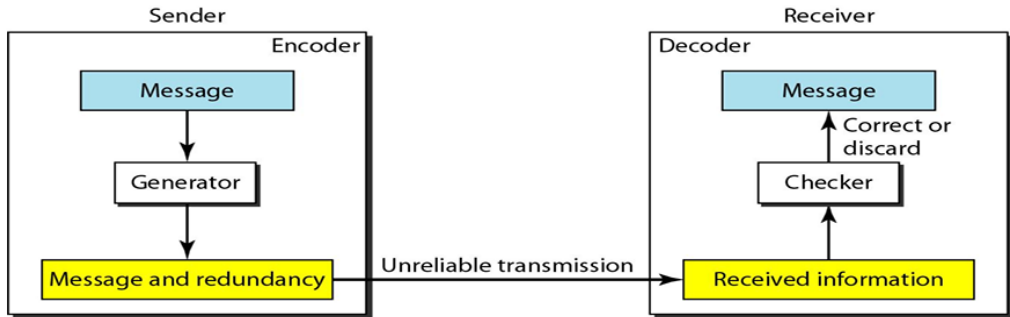
Redundancy

- The central concept in detecting or correcting errors is Redundancy.
- To detect or correct errors, we need to send extra (redundant) bits with data. This technique is called **Redundancy**.
- These extra bits are discarded as soon as the accuracy of the transmission has been determined.
- Redundancy is achieved through various coding schemes.



Coding Schemes

- The sender adds redundant bits through a process that creates a relationship between redundant bits and the actual data bits.
- The receiver checks the relationships between the two sets of bits to detect or correct the errors.



Error Detection Schemes

As there are many error detection schemes, we will focus on a few fundamental and widely used ones in this course:

- **Single Parity Check**
- **Two-Dimensional Parity Check**
- **Checksum**
- **Cyclic Redundancy Check (CRC)**



Single-Parity Check

- Adds one extra bit (parity bit) to a data unit.
- Ensures total number of 1s is even (even parity) or odd (odd parity).
- **Example:**
 - **Even Parity**
 - Data: 1010101 (4 ones) $\Rightarrow$ Parity bit = 0 (even parity)
 - Transmitted: 10101010
 - **Odd Parity**
 - Data: 1010101 (4 ones) $\Rightarrow$ Parity bit = 1 (odd parity)
 - Transmitted: 10101011
- **Can detect:** All single-bit errors.
- **Cannot detect:** All burst errors (if the number of flipped bits is even).



Single-Parity Check: Examples (Even Parity)

- **Example 01**

- Data: 1010101 (4 ones)
- Parity Bit: 0 (to keep count of 1s even)
- Transmitted: 10101010
- Received: 10100010 $\Rightarrow$ Even number of 1s $\Rightarrow$ **No error**

- **Example 02**

- Transmitted: 10101010
- Suppose a bit is flipped: 11101010
- Received: 5 ones (odd) $\Rightarrow$ **Error detected**

- **Example 03**

- Transmitted: 10101010
- Suppose two bits are flipped: 11101110
- Received: 6 ones (even) $\Rightarrow$ **Error not detected!**



Two-Dimensional Parity Check

- Improves error detection by adding parity bits to both rows and columns.
- Can detect and correct single-bit errors.

Original data: 10 11

1	0
1	1

After adding row and column parity (even parity):

1	0		1
1	1		0

0	1		1

Final Data: 10 11 101 011



Two-Dimensional Parity: Example

Original Data: 1011 0101 1110 0011

1	0	1	1
0	1	0	1
1	1	1	0
0	0	1	1

Adding parity bits: (even parity)

1	0	1	1		1
0	1	0	1		0
1	1	1	0		1
0	0	1	1		0

0	0	1	1		0
---	---	---	---	--	---

Final Data: 1011 0101 1110 0011 10100 00110



Two-Dimensional Parity: Error Detection

- **Single-bit errors:** Easily detected — row and column parity both mismatch.
- **Two-bit errors:**
 - If the two bits are in *different rows and columns*, both row and column parities will mismatch → detected.
 - If the two errors occur in the *same row and same column*, the parities may cancel out → not detected.
- **Three-bit errors:**
 - Always change an odd number of row and/or column parities.
 - Cannot "cancel out" completely → will be detected.
- **Limitation:**
 - 2D parity can detect (but not correct) up to 3-bit errors.
 - Cannot detect all 4-bit errors or even some 2-bit errors.



Checksum

- Sender and receiver agree on a block size (e.g., 4 bits, 8 bits, etc.).
- Data is divided into equal segments and summed.
- The sender transmits the complement of this sum.
- The receiver verifies by repeating the process; mismatches indicate errors.

Analogy:

Data: (7, 3) $\rightarrow$ Sum = 10 $\rightarrow$ Checksum = -10

Send: (7, 3, -10) $\rightarrow$ Total = 0 = No Error



One's Complement Arithmetic

- **One's complement** flips each bit:
 - 0 becomes 1, 1 becomes 0.
 - Example: $1010 \rightarrow 0101$
- **One's Complement Addition:**
 - Add binary numbers normally.
 - If a carry is generated, **wrap it around** and add to the least significant bit.
- **Example:**
 - Add: $1101 + 1011 = 11000 \rightarrow$ drop carry bit: $1000 + 1 = 1001$
 - Final sum = 1001
- **Checksum:** Take 1's complement of this sum.



Checksum: Example

Original Data: 1001 0101 1100 0011

1001

0101

1100

0011

Step 1: Add all blocks using 1's complement addition

1001 + 0101 = 1110

1110 + 1100 = 1 1010 → wrap around: 1010 + 1 = 1011

1011 + 0011 = 1110

Step 2: Take 1's complement of 1110 → Checksum = 0001

Final Data: 1001 0101 1100 0011 0001



Checksum: Example

Verification of Received Data: 1001 0101 1100 0011 0001

1001

0101

1100

0011

0001 (checksum)

Step 1: Add all blocks with checksum

Same as before: Final sum = 1110

1110 + 0001 = 1111

Step 2: Take 1's complement of 1111 → 0000 → No error

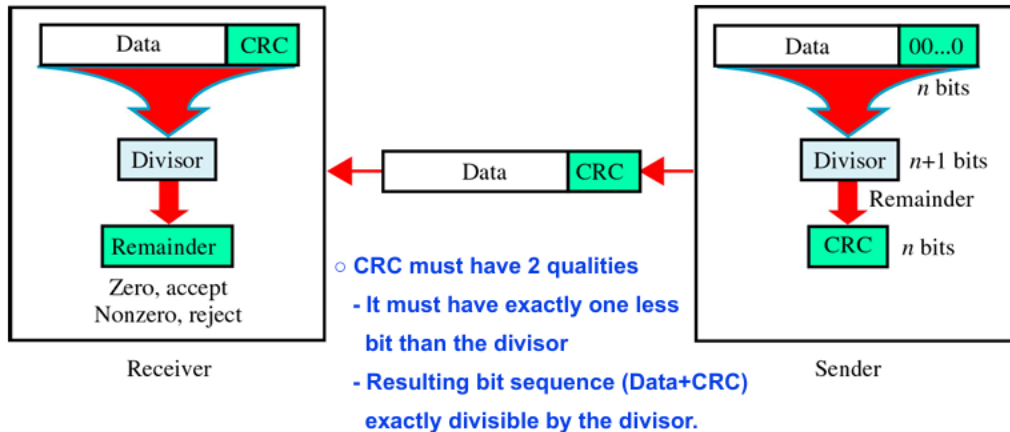


Cyclic Redundancy Check (CRC)

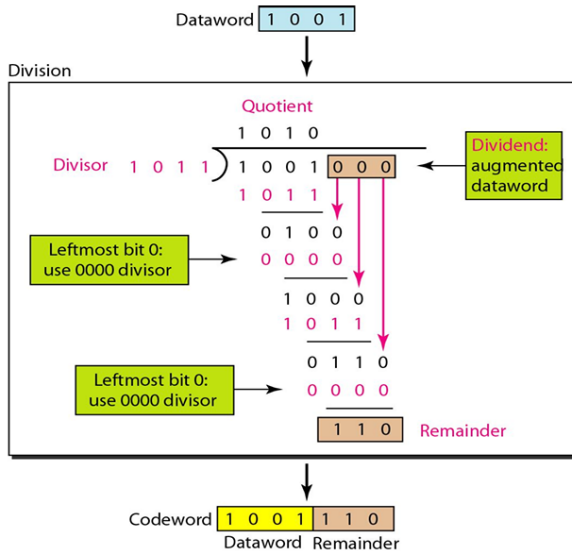
- CRC (Cyclic Redundancy Check) is a powerful error detection technique.
- It treats data as a polynomial and divides (binary XOR) it by a fixed generator polynomial.
- The remainder from this division is the *CRC code*.
- The CRC checksum is appended to the data before transmission.
- At the receiver, data + CRC is divided by the same polynomial.
- If remainder = 0, data is assumed error-free; else error detected.
- CRC can detect burst errors and multiple bit errors better than parity or checksum.



CRC



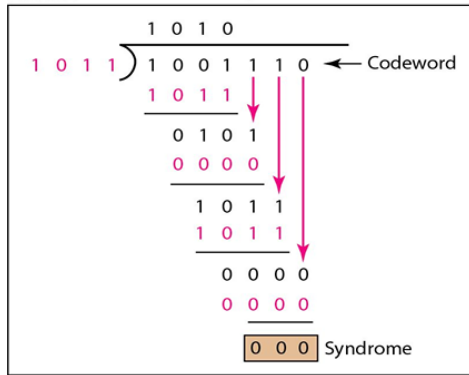
CRC: Example



CRC: Example

Codeword **1 0 0 1** **1 1 0**

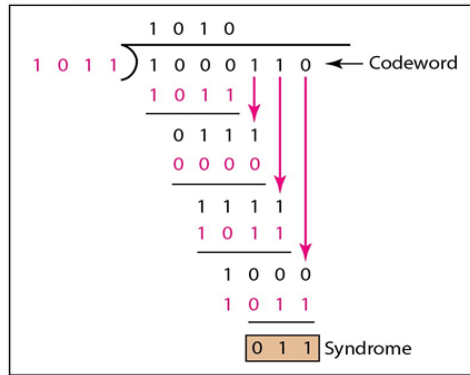
Division



Dataword accepted **1 0 0 1**

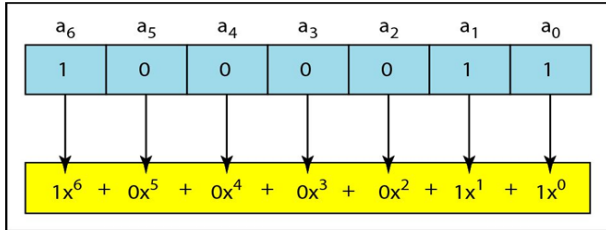
Codeword **1 0 0 0** **1 1 0**

Division

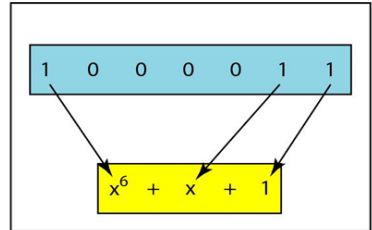


Dataword discarded

CRC: Polynomial Representation



a. Binary pattern and polynomial



b. Short form

CRC: Standard

<i>Name</i>	<i>Polynomial</i>	<i>Application</i>
CRC-8	$x^8 + x^2 + x + 1$	ATM header
CRC-10	$x^{10} + x^9 + x^5 + x^4 + x^2 + 1$	ATM AAL
CRC-16	$x^{16} + x^{12} + x^5 + 1$	HDLC
CRC-32	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$	LANs



CRC

Advantages:

- Very good performance
- Easily be implementation in hardware and software
- Faster



Error Correction: Hamming Code

- Error detection identifies data corruption during transmission or storage.
- Error correction goes a step further: fixes the error without retransmission.
- Hamming Code is an error correction scheme that:
 - Detects and corrects single-bit errors.
 - Locates the error position using parity bits.



Hamming Code: Example

- Hamming code $C(7,4)$ adds parity bits to 4 data bits to make a 7-bit codeword.
- Bit positions: 1-based index – 7 6 5 4 3 2 1
- Number of parity bits, m , is determined by:

$$2^m \geq m + d + 1$$

- For $d = 4$, minimum m satisfying the formula is $m = 3$:

$$2^3 = 8 \geq 3 + 4 + 1 = 8$$

- Parity bits at positions: 1, 2, 4 (powers of 2)
- Data bits placed in positions: 7, 6, 5, 3



Hamming Code: Example

- Parity bits are calculated based on binary representation of positions.

Position	Binary	Covered by
1	001	p_1
2	010	p_2
3	011	p_1, p_2
4	100	p_3
5	101	p_1, p_3
6	110	p_2, p_3
7	111	p_1, p_2, p_3

$$p_1 = b_3 \oplus b_5 \oplus b_7$$

$$p_2 = b_3 \oplus b_6 \oplus b_7$$

$$p_3 = b_5 \oplus b_6 \oplus b_7$$



Hamming Code: Example

Example: Data = 1010

- Place data bits: 1 0 1 _ 0 _ _
- Compute parity bits:

$$p_1 = b_3 \oplus b_5 \oplus b_7 = 0 \oplus 1 \oplus 1 = 0$$

$$p_2 = b_3 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 1 = 1$$

$$p_3 = b_5 \oplus b_6 \oplus b_7 = 1 \oplus 0 \oplus 1 = 0$$

Encoded message: 1 0 1 0 0 1 0

(XOR calculation is as same as calculating even parity)



Hamming Code: Example

[Assuming no error]

- Received message: 1 0 1 0 0 1 0
- Recalculate parity checks:

$$S_1 = b_1 \oplus b_3 \oplus b_5 \oplus b_7 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

$$S_2 = b_2 \oplus b_3 \oplus b_6 \oplus b_7 = 1 \oplus 0 \oplus 0 \oplus 1 = 0$$

$$S_3 = b_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$$

- Syndrome = $S_3 S_2 S_1 = 000_2 = 0 \rightarrow$ No Error
- Extract data bits: positions 3,5,6,7 $\rightarrow$ 1 0 1 0



Hamming Code: Example

[Assuming an error in 5th bit]

- Received message: 1 0 0 0 1 0
- Recalculate parity checks:

$$S_1 = b_1 \oplus b_3 \oplus b_5 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$$

$$S_2 = b_2 \oplus b_3 \oplus b_6 \oplus b_7 = 1 \oplus 0 \oplus 0 \oplus 1 = 0$$

$$S_3 = b_4 \oplus b_5 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 0 \oplus 1 = 1$$

- Syndrome = $S_3S_2S_1 = 101_2 = 5 \rightarrow$ Error in bit 5
- Flip bit 5: Corrected = 1 0 1 0 0 1
- Extract data bits: positions 3,5,6,7 $\rightarrow$ 1 0 1 0



References

- Data Communication and Networking (4th Ed) - Behrouz A. Forouzan
Chapter 10: 10.1 - 10.5



Thank You

